RAPPORT DE STAGE D'INITIATION

Développement d'une Application Web de Gestion des Comptes Rendus d'Activité

APPLICATION CRATECH

Réalisé par :	**AMEZIANE Oumaima**
Filière :	Génie Informatique
Encadré par :	Mr. KARIM Mohammed-Amine
Encadrant pédagogique :	Mr. ABTOY Anouar

Année Universitaire 2024-2025

Remerciements

Je tiens à exprimer mes sincères remerciements à mon encadrant **KARIM Mohammed-Amine** au sein de l'entreprise **AGILE SQUARE** pour l'accompagnement précieux qu'il m'a accordé tout au long de ce stage d'initiation réalisé à distance. Sa disponibilité, son professionnalisme et la pertinence de ses orientations ont joué un rôle essentiel dans le bon déroulement de cette expérience.

Grâce à son expertise et à son sens du partage, j'ai pu mieux comprendre les exigences du milieu professionnel, approfondir mes connaissances techniques et développer de nouvelles compétences.

Ce stage a constitué une première immersion enrichissante dans le monde professionnel. Il m'a offert l'opportunité de mettre en pratique les acquis théoriques, de m'adapter à un environnement de travail structuré et de renforcer mon sens de l'autonomie et de la responsabilité.

Je remercie également l'École Nationale des Sciences Appliquées de Tétouan, ainsi que l'ensemble du corps enseignant, pour la qualité de la formation dispensée. Les enseignements reçus ont représenté un socle fondamental pour la réussite de cette expérience et pour la construction de mon parcours académique .

Résumé

Ce rapport présente le travail effectué durant mon stage d'initiation consistant à concevoir et développer CRATECH, une application web moderne de gestion des Comptes Rendus d'Activités (CRA).

L'objectif principal était de créer une solution permettant aux employés et sous-traitants de saisir mensuellement leurs activités professionnelles, tout en offrant aux administrateurs et validateurs des outils de supervision et de validation efficaces. Le projet s'inscrit dans une démarche d'automatisation et d'optimisation des processus administratifs.

Pour mener à bien ce projet, j'ai utilisé une pile technologique moderne comprenant Laravel 12.0 pour le backend, Tailwind CSS , HTML et JavaScript pour le frontend, ainsi que MySQL pour la gestion des données. L'authentification sécurisée a été implémentée via Keycloak avec le protocole OpenID Connect.

L'application finale offre des fonctionnalités complètes incluant la saisie de CRA avec auto-sauvegarde, un système de validation multi-niveaux, des tableaux de bord interactifs avec visualisations graphiques, ainsi que l'export de rapports au format Excel.

Ce stage m'a permis d'acquérir une expérience concrète en développement web full-stack et de renforcer mes compétences techniques et organisationnelles.

Table des matières

Remerciements **1**

Résumé **2**

1 Introduction Générale **6**
1.1 Contexte et Problématique 6
1.2 Objectifs du Projet 6

2 Analyse et Conception du Projet **7**
2.1 Spécification des Besoins 7
2.1.1 Besoins Fonctionnels 7
2.1.2 Besoins Non Fonctionnels 8
2.2 Modélisation UML 8
2.2.1 Diagramme des Cas d'Utilisation 8
2.2.2 Diagramme de Classes 9
2.2.3 Diagramme de séquence 10
2.3 Architecture du Système 11
2.3.1 Architecture Globale 11
2.4 Schéma de Base de Données 12
2.4.1 Légende 12
2.4.2 Schéma relationnel 12
2.4.3 Relations entre les entités 13

3 Réalisation Technique **14**
3.1 Environnement de Développement 14
3.1.1 Outils et Technologies 14
3.1.2 Configuration du Projet 14
3.2 Implémentation des Modules 14
3.2.1 Module d'Authentification Hybride et Intégré 14
3.2.2 Module de Gestion des Utilisateurs 15
3.2.3 Module de Saisie des CRA 15
3.2.4 Module de Validation et Reporting 16
3.2.5 Export des Données 16

3.3 Sécurité Implémentée . . . 16
3.3.1 Protection contre les attaques CSRF . . . 16
3.3.2 Filtrage et Validation des Données . . . 16
3.3.3 Contrôle d'Accès Basé sur les Rôles (RBAC) . . . 17
3.4 Défis Techniques Rencontrés . . . 17
3.4.1 Orchestration de l'Authentification Hybride . . . 17
3.4.2 Indicateur de Synchronisation Temps Réel (CRA) . . . 17
3.4.3 Optimisation de la Data-Visualisation . . . 17

4 Résultats et Évaluation **19**
4.1 Bilan des fonctionnalités réalisées . . . 19
4.2 Guide d'Utilisation de l'Application . . . 19
4.2.1 Interface Utilisateur . . . 19
4.2.2 Interface Administrateur . . . 22
4.2.3 Interface Validateur . . . 23
4.3 Évaluation critique des résultats . . . 24
4.3.1 Performance de l'architecture technique . . . 24
4.3.2 Impact sur le workflow métier . . . 24
4.4 Compétences Acquises . . . 25
4.4.1 Compétences Techniques . . . 25
4.4.2 Compétences Transversales . . . 25
4.5 Conclusion Générale . . . 26

5 Documentation Technique **27**
5.1 Installation et Déploiement . . . 27
5.1.1 Prérequis . . . 27
5.1.2 Procédure d'Installation . . . 27
5.1.3 Fichier d'Environnement . . . 27

Table des figures

2.1 Diagramme des cas d'utilisation de CRATECH . . . 9
2.2 Diagramme de classes de CRATECH . . . 10
2.3 Diagramme de séquence – Processus global de gestion des CRA . . . 11

4.1 Interface de connexion . . . 20
4.2 Interface des Uses dans Keycloak . . . 20

4.3 Tableau de bord employé et sous-traitant 20
4.4 Tableau de bord employé et sous-traitant 21
4.5 Historique des CRA 21
4.6 Tableau de bord administrateur 22
4.7 Interface de gestion des utilisateurs 22
4.8 Interface de gestion des activites 23
4.9 Liste des CRA soumis 23
4.10 Interface de validation des CRA 24

Liste des tableaux

3.1 Technologies utilisées pour le développement 14

Chapitre 1

Introduction Générale

1.1 Contexte et Problématique

Dans un environnement professionnel en constante évolution, la gestion efficace du temps de travail et le suivi des activités des employés constituent des enjeux majeurs. La digitalisation des processus administratifs est devenue une nécessité pour améliorer la productivité, réduire les erreurs et faciliter la prise de décision.

La gestion manuelle des Comptes Rendus d'Activités (CRA) présente plusieurs problématiques : perte de temps considérable pour la saisie et la consolidation des données, risques d'erreurs humaines dans le reporting, difficulté de traçabilité et d'archivage des informations, processus de validation long et complexe, et manque de visibilité en temps réel sur l'activité des équipes.

1.2 Objectifs du Projet

Le projet CRATECH a pour objectifs principaux de concevoir et de développer une application web complète dédiée à la gestion des comptes rendus d'activité (CRA). Il vise à intégrer un système d'authentification sécurisé basé sur Keycloak, ainsi qu'à proposer une interface intuitive et ergonomique, adaptée aux différents profils d'utilisateurs. Le projet comprend également la mise en place de tableaux de bord offrant des visualisations graphiques pertinentes, la génération automatique de rapports exportables, et l'application des bonnes pratiques en matière de développement et de sécurité.

Chapitre 2

Analyse et Conception du Projet

2.1 Spécification des Besoins

2.1.1 Besoins Fonctionnels

L'analyse des besoins a permis d'identifier les fonctionnalités principales suivantes :

Gestion de l'authentification :

1. Connexion sécurisée via Keycloak (SSO)
2. Authentification OpenID Connect
3. Gestion des sessions utilisateurs

Gestion des utilisateurs :

1. Création et modification des comptes
2. Affectation des rôles (Admin, Employé, Sous-traitant, Validateur)
3. Activation et désactivation des comptes

Gestion des activités :

1. Création de projets et activités
2. Attribution de codes projets
3. Association activités-utilisateurs

Gestion des CRA :

1. Saisie mensuelle des jours travaillés
2. Enregistrement par activité
3. Auto-sauvegarde en temps réel (brouillon)
4. Soumission pour validation

Workflow de validation :

1. Liste des CRA en attente de validation
2. Visualisation détaillée du CRA
3. Validation ou rejet du CRA
4. Notifications aux utilisateurs concernés

Tableaux de bord et reporting :

1. Dashboard collaborateur avec statut du CRA en cours
2. Dashboard administrateur avec statistiques globales mise a jour en temps réel
3. Graphiques d'évolution des validations
4. Export Excel des CRA validés

2.1.2 Besoins Non Fonctionnels

Les exigences non fonctionnelles incluent :

Performance :

1. Temps de réponse inférieur à 2 secondes
2. Optimisation des requêtes SQL
3. Mise en cache des données fréquemment consultées

Sécurité :

1. Protection CSRF native de Laravel
2. Validation et sanitization des entrées utilisateur
3. Gestion stricte des permissions (RBAC - Role Based Acces Control)
4. Authentification centralisée via Keycloak

2.2 Modélisation UML

2.2.1 Diagramme des Cas d'Utilisation

Le diagramme des cas d'utilisation identifie les acteurs du système et leurs interactions avec l'application CRATECH.

Les acteurs identifiés sont :

— **Employé / Sous-traitant** : Acteur principal du système, il saisit, modifie, sauvegarde et soumet ses CRA (Compte Rendu d'Activité). Il peut également consulter son espace personnel et télécharger ses rapports CRA.

— **Validateur** : Responsable de la validation des CRA soumis par les employés. Il peut consulter les CRA, les valider ou les refuser. En cas d'indisponibilité, une procédure exceptionnelle permet à l'administrateur d'intervenir.

— **Administrateur** : Supervise l'ensemble du système. Il gère les utilisateurs, affecte les validateurs, valide les CRA en cas exceptionnel, administre les paramètres du système et peut générer des rapports CRA.

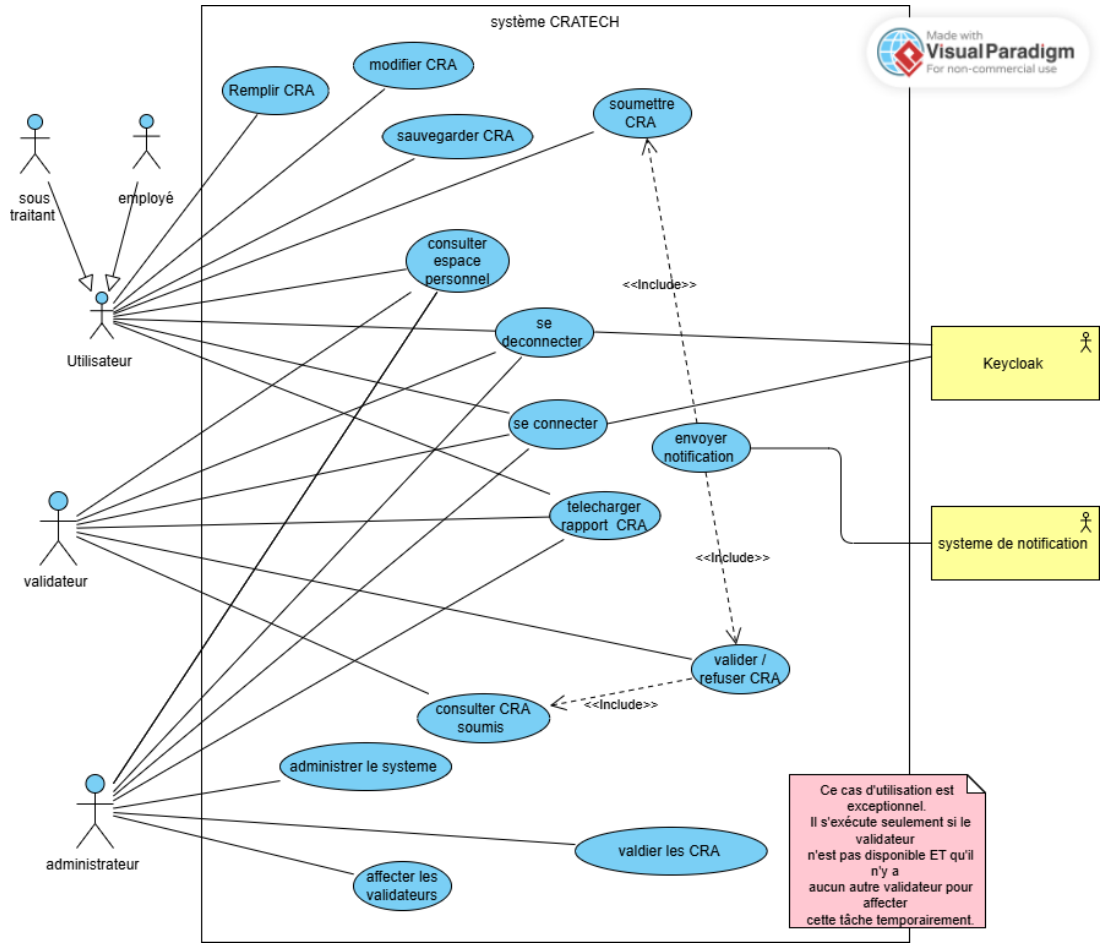


FIGURE 2.1 – Diagramme des cas d'utilisation de CRATECH

— **Keycloak** : Système d'authentification externe utilisé pour la gestion sécurisée des connexions (login/logout) des utilisateurs.

— **Système de notification** : Composant technique intégré qui envoie automatiquement des notifications lors de la soumission ou de la validation/refus d'un CRA.

2.2.2 Diagramme de Classes

Le diagramme de classes représente la structure statique de l'application et les relations entre les différentes entités.

Les classes principales sont :

— Utilisateur : Représente un compte utilisateur

— Activité : Projet ou tâche sur lequel on peut imputer du temps

— CRA : Compte Rendu d'Activité mensuel

— JourActivite : Détail d'imputation pour un jour donné

— UserAct : Association entre utilisateur et activité

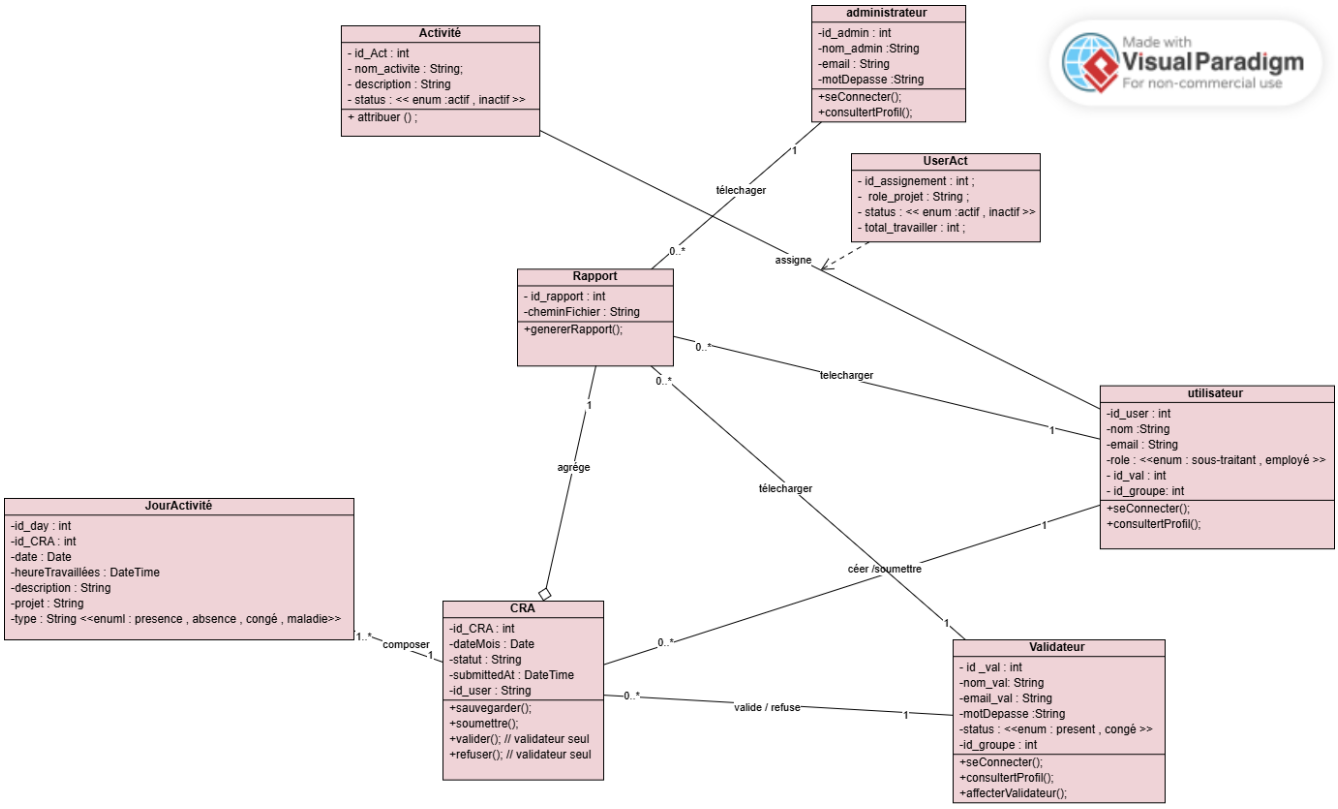


FIGURE 2.2 – Diagramme de classes de CRATECH

2.2.3 Diagramme de séquence

Le diagramme de séquence illustre les interactions dynamiques entre les différents acteurs du système et ses composants internes lors du processus global de gestion des Comptes Rendus d'Activité (CRA). Il met en évidence l'enchaînement temporel des échanges, depuis l'authentification de l'utilisateur jusqu'à la saisie, la soumission et la validation des CRA, ainsi que la gestion des utilisateurs par l'administrateur.

Ce diagramme permet notamment de visualiser l'intégration du mécanisme d'authentification via Keycloak, la création automatique des utilisateurs (JIT provisioning), les interactions entre les interfaces utilisateur et les contrôleurs applicatifs, ainsi que les échanges avec la base de données et le service de messagerie lors des phases de validation ou de rejet des CRA.

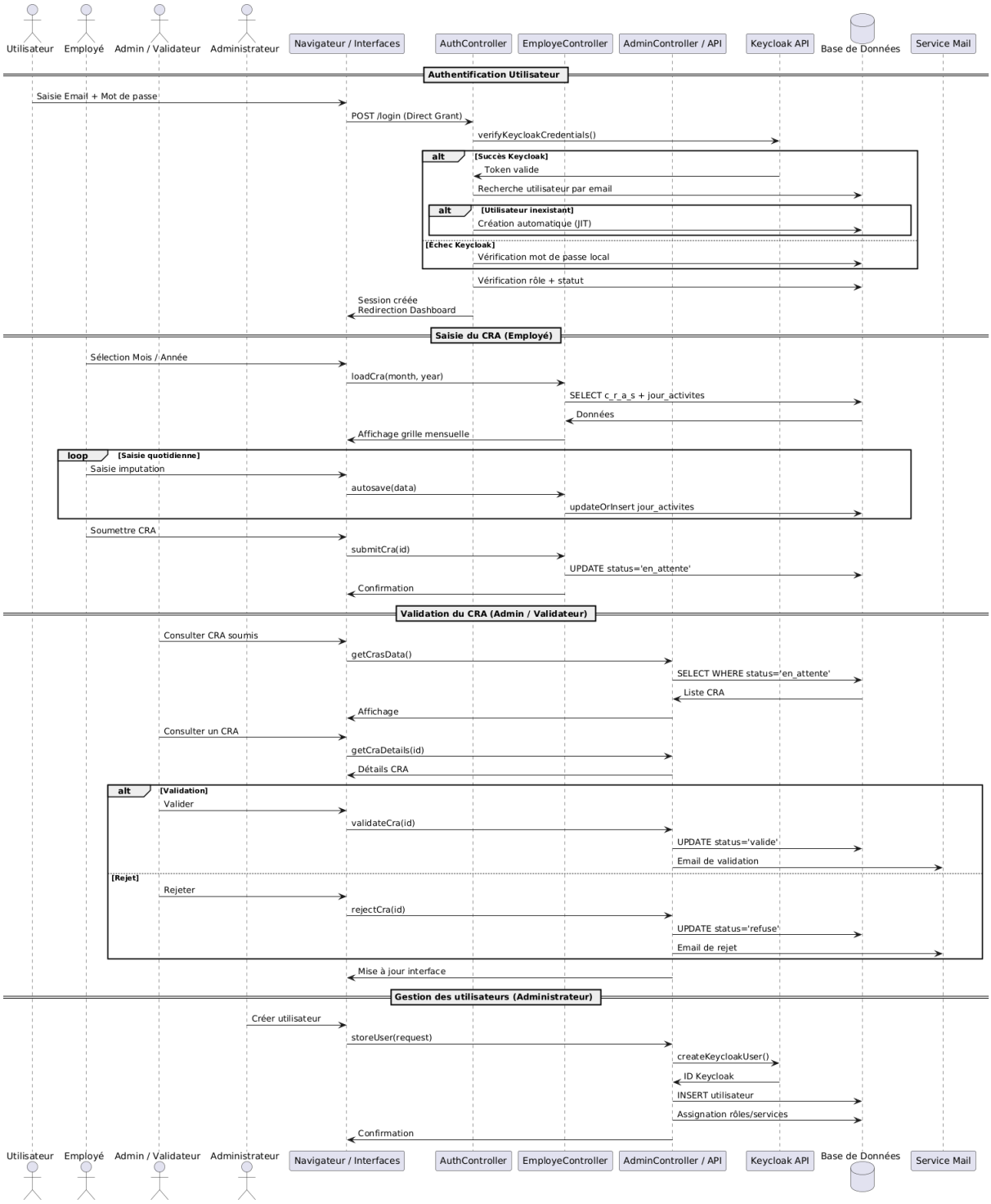


FIGURE 2.3 – Diagramme de séquence – Processus global de gestion des CRA

2.3 Architecture du Système

2.3.1 Architecture Globale

CRATECH adopte une architecture MVC (Model-View-Controller) classique offerte par Laravel, avec une séparation claire entre la logique métier et la présentation.

L'architecture se compose de trois couches principales :

Couche Présentation (Frontend) :

— Vite.js 6 pour le bundling des assets

— Tailwind CSS 4 pour le styling

— JavaScript (ESM) pour l'interactivité

— Chart.js pour les visualisations

— Axios pour les appels API asynchrones

Couche Métier (Backend) :

— Laravel 12.0 avec architecture MVC

— Contrôleurs pour la logique métier

— Modèles Eloquent pour l'accès aux données

— Middlewares pour la sécurité et l'authentification

— API RESTful pour la communication frontend/backend

Couche Données :

— MySQL/MariaDB comme SGBD

— Eloquent ORM pour les requêtes

— Migrations Laravel pour la gestion du schéma

2.4 Schéma de Base de Données

La base de données de l'application CRATECH a été conçue selon un modèle relationnel afin de garantir la cohérence, l'intégrité et la traçabilité des informations liées à la gestion des Comptes Rendus d'Activité (CRA). Elle repose sur plusieurs entités principales interconnectées par des clés primaires et étrangères.

2.4.1 Légende

— **PK** : Clé primaire (Primary Key)

— **FK** : Clé étrangère (Foreign Key)

2.4.2 Schéma relationnel

— **UTILISATEURS**
`(id_user : PK, nom_user, email_user, motdepasse_user, role, status, keycloak_id, id_validateur : FK)`

— **ACTIVITÉS**
`(id_activite : PK, nom_act, description, status)`

— **C_R_A_S**
`(id_CRA : PK, id_user : FK, dateMois, status, submittedAt)`

— **JOUR_ACTIVITES**
`(id_day : PK, id_CRA : FK, id_activite : FK, date, type, description)`

— **USER__ACTS**
`(id_assignement : PK, id_user : FK, id_activite : FK, role_projet, status, total_travaille)`

— **RAPPORT_MENSUELS**
`(rapport_mensuel_id : PK, id_CRA : FK, chemin_fichier)`

2.4.3 Relations entre les entités

Les principales relations du schéma relationnel sont les suivantes :

— Un **utilisateur** peut créer plusieurs **CRA** (relation un-à-plusieurs).

— Chaque **CRA** regroupe plusieurs **journées d'activité**.

— Les **utilisateurs** et les **activités** sont liés par une relation plusieurs-à-plusieurs, matérialisée par la table **USER__ACTS**.

— Chaque **journée d'activité** est associée à une **activité** précise.

— Les **rapports mensuels** sont générés à partir des CRA et des utilisateurs.

Chapitre 3

Réalisation Technique

3.1 Environnement de Développement

3.1.1 Outils et Technologies

Le tableau suivant présente les principaux outils et technologies utilisés :

Catégorie	**Outil/Technologie**
IDE	Visual Studio Code
Gestionnaire de versions	Git
Backend Framework	Laravel 12.0
Frontend Framework	Tailwind CSS 4, Vite.js 6
Langage de base	CSS Vanilla, HTML , JS
Base de données	MySQL 8.0
Authentification	Keycloak 24.0
Visualisation de données	Chart.js 4

TABLE 3.1 – Technologies utilisées pour le développement

3.1.2 Configuration du Projet

L'installation et la configuration du projet ont suivi les étapes suivantes :

1. Installation de Laravel via Composer
2. Configuration de la base de données dans le fichier .env
3. Installation des dépendances frontend avec npm
4. Configuration de l'intégration Keycloak
5. Mise en place de la structure de dossiers Laravel

3.2 Implémentation des Modules

3.2.1 Module d'Authentification Hybride et Intégré

L'authentification a été implémentée de manière hybride et totalement transparente. Le système utilise un formulaire de connexion unique et orchestre la validation entre

Keycloak (via l'API *Direct Grant*) et la base de données locale en arrière-plan.

Le flux d'authentification réel comprend :

1. L'utilisateur saisit ses identifiants sur une interface de connexion unifiée.
2. Le backend Laravel tente une validation auprès de l'API Keycloak (`openid-connect/token`).
3. En cas de succès Keycloak, le compte local est synchronisé ou créé automatiquement (*Just-In-Time Provisioning*).
4. En cas d'échec Keycloak, le système bascule sur la vérification locale du mot de passe haché (BCrypt).
5. La session Laravel est initialisée avec redirection dynamique selon le rôle détecté.

La configuration Keycloak centralisée dans Laravel :

```
// config/services.php
'keycloak' => [
    'client_id' => env('KEYCLOAK_CLIENT_ID'),
    'client_secret' => env('KEYCLOAK_CLIENT_SECRET'),
    'base_url' => env('KEYCLOAK_BASE_URL'),
    'realms' => env('KEYCLOAK_REALM'),
],
```

3.2.2 Module de Gestion des Utilisateurs

Ce module permet aux administrateurs de piloter les accès :

— Liste paginée avec recherche dynamique.

— Gestion des rôles (Admin, Employé, Sous-traitant, Validateur).

— **Affectation hiérarchique** : Liaison directe entre un employé et son validateur.

— Contrôle du statut (Activation/Désactivation instantanée).

3.2.3 Module de Saisie des CRA

C'est le composant métier principal. Il repose sur une architecture *Single Page* pour une fluidité maximale.

1. **Grille Dynamique** : Générée en JavaScript selon le nombre de jours du mois sélectionné.
2. **Auto-sauvegarde événementielle** : Chaque modification de cellule déclenche un appel asynchrone (`fetch`) vers l'API `/api/cra/autosave`.
3. **Gestion des statuts** : Verrouillage automatique de la grille dès que le CRA est soumis.

Extrait de la logique d'auto-sauvegarde (Frontend) :

```
// resources/js/employe.js
async autoSave() {
    if (!this.currentCRAId) return;
    const payload = { cra_id: this.currentCRAId, data: this.craData };
    await fetch('/employe/cra/autosave', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json', 'X-CSRF-TOKEN':
            this.csrf.value },
        body: JSON.stringify(payload)
    });
}
```

3.2.4 Module de Validation et Reporting

Les validateurs disposent d'un tableau de bord de supervision :

— Visualisation des CRA en attente des employés rattachés.

— Comparaison des données et validation/rejet en un clic.

— **Reporting visuel** : Utilisation de *Chart.js* pour l'analyse des temps passés et l'évolution des validations sur les 6 derniers mois.

3.2.5 Export des Données

L'export Excel a été optimisé via une génération **côté client**. Le système construit une structure XML/HTML compatible Excel en mémoire, incluant le formatage (couleurs pour les week-ends, totaux), puis déclenche le téléchargement via un *Blob* JavaScript. Cette approche réduit la charge serveur et offre un téléchargement instantané pour l'utilisateur.

3.3 Sécurité Implémentée

3.3.1 Protection contre les attaques CSRF

Cratech utilise le mécanisme natif de Laravel pour prévenir les attaques *Cross-Site Request Forgery*. Un jeton de session unique est généré et vérifié pour chaque requête `POST`, `PUT` ou `DELETE`. Pour les composants JavaScript, une route dédiée `/csrf-token` a été implémentée pour synchroniser dynamiquement les en-têtes `X-CSRF-TOKEN`.

3.3.2 Filtrage et Validation des Données

L'intégrité des données est assurée par des validateurs Laravel rigoureux sur chaque point d'entrée API.

```php
// Exemple : Validation lors de la cr ation d'un utilisateur
$request->validate([
    'nom_user' => 'required|string|max:255',
    'email_user' => 'required|email|unique:utilisateurs,email_user',
    'password' => 'required|string|min:8',
    'role' => 'required|in:administrateur,validateur,employe,sous-
        traitant',
]);
```

3.3.3 Contrôle d'Accès Basé sur les Rôles (RBAC)

La sécurité des accès repose sur un middleware personnalisé `CheckRole` qui intercepte les requêtes et vérifie les privilèges en session avant d'accorder l'accès aux ressources sensibles (Admin, Validation, Imputation).

3.4 Défis Techniques Rencontrés

3.4.1 Orchestration de l'Authentification Hybride

Problème : Concilier l'authentification Keycloak (SSO) et le stockage local des utilisateurs sans imposer de redirection visuelle à l'utilisateur.
Solution : Implémentation d'une logique "Transparent Failover". Le système interroge d'abord Keycloak via Direct Grant ; en cas d'échec, il valide les identifiants localement. Ce défi a nécessité la mise en place d'un *Just-In-Time Provisioning* pour créer automatiquement les comptes Keycloak validés dans la base locale.

3.4.2 Indicateur de Synchronisation Temps Réel (CRA)

Problème : Assurer la sauvegarde des données sans impacter la fluidité de la saisie (latence réseau).
Solution : Utilisation de la programmation événementielle en JavaScript. Chaque cellule modifiée déclenche un appel asynchrone léger. Un système de notifications visuelles (*Toast notifications*) informe l'utilisateur du succès de la synchronisation instantanée.

3.4.3 Optimisation de la Data-Visualisation

Problème : Affichage dynamique de statistiques complexes sur le dashboard administrateur.
Solution : Centralisation des statistiques via des APIs JSON dédiées et exploitation de

la bibliothèque *Chart.js*. L'optimisation a porté sur la réduction du nombre de requêtes SQL groupées via l'Eloquent ORM pour garantir un chargement rapide des graphiques.

Chapitre 4

Résultats et Évaluation

4.1 Bilan des fonctionnalités réalisées

Le développement de l'application CRATECH a abouti à une plateforme opérationnelle couvrant l'intégralité du cycle de vie des comptes rendus d'activité.

L'implémentation de l'authentification hybride intégrée constitue le socle sécuritaire du projet, permettant une validation transparente des identifiants via l'API Keycloak tout en assurant un provisionnement automatique des comptes en base locale. Les administrateurs disposent désormais d'un contrôle granulaire sur la gestion des utilisateurs et des activités, incluant un système de permissions strict basé sur les rôles (RBAC).

Le module de saisie des CRA offre une expérience utilisateur fluide grâce à une grille dynamique qui détecte automatiquement la structure du mois et les week-ends. Cette interface est couplée à une logique d'auto-sauvegarde asynchrone (AJAX) garantissant l'intégrité des données sans interrompre le travail de l'employé. Le workflow est complété par un système de validation multi-niveaux où les validateurs peuvent réviser, valider ou rejeter les rapports, déclenchant ainsi des notifications par email automatiques. Enfin, la plateforme intègre des outils décisionnels avancés, notamment des dashboards interactifs propulsés par Chart.js et un module d'export Excel dynamique généré côté client pour une réactivité maximale.

4.2 Guide d'Utilisation de l'Application

Cette section présente le guide d'utilisation de l'application **CRATECH**. Elle décrit, de manière structurée, les différentes interfaces mises à disposition des utilisateurs ainsi que les fonctionnalités associées à chacune d'elles.

4.2.1 Interface Utilisateur

Interface de Connexion

Cette interface permet à l'utilisateur d'accéder à l'application via un portail de connexion sécurisé. L'authentification est effectuée à l'aide d'identifiants locaux et via le système d'authentification centralisé Keycloak (système d'authentification hybride) .

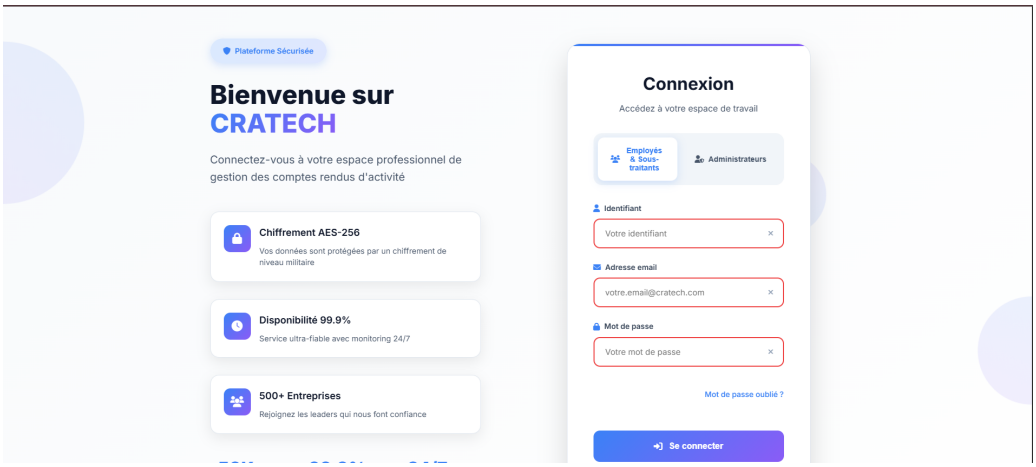


FIGURE 4.1 – Interface de connexion

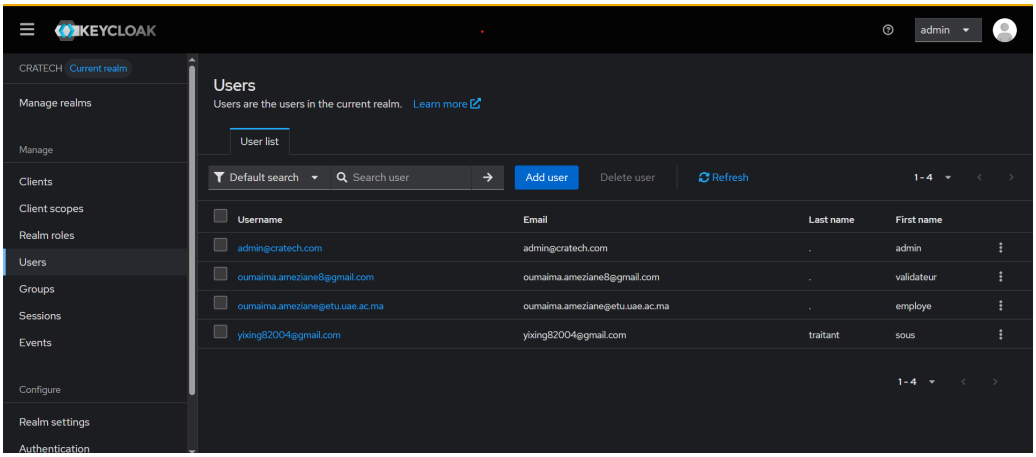


FIGURE 4.2 – Interface des Uses dans Keycloak

.

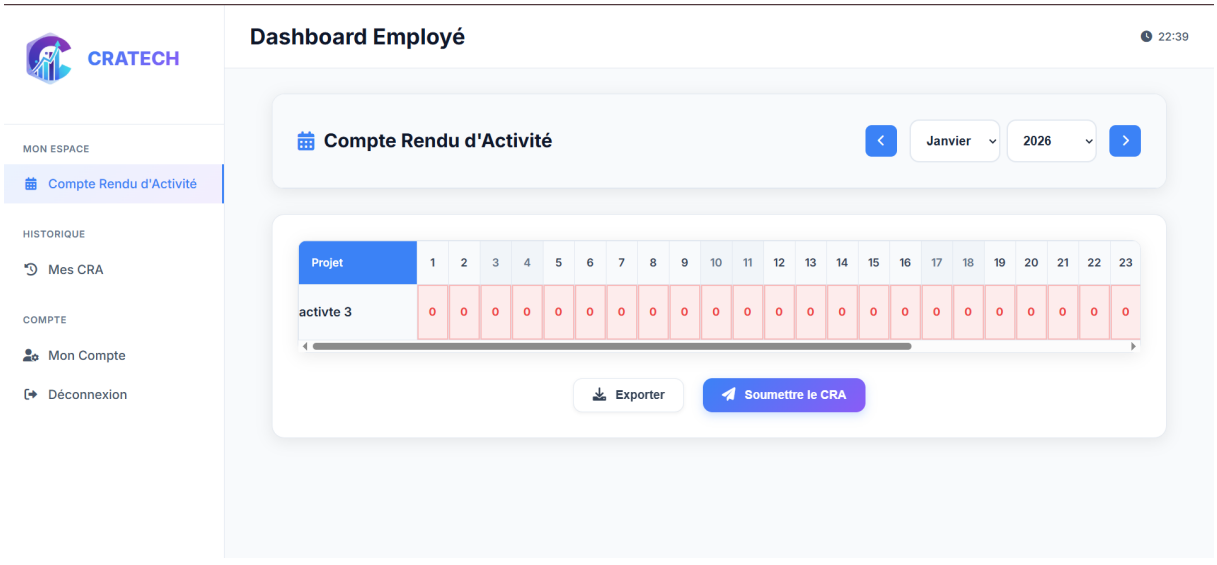


FIGURE 4.3 – Tableau de bord employé et sous-traitant

Saisie du Compte Rendu d'Activité

Cette interface permet à l'utilisateur de saisir ses activités journalières à travers une grille d'imputation dynamique. Les données sont sauvegardées automatiquement afin de garantir la fiabilité de la saisie.

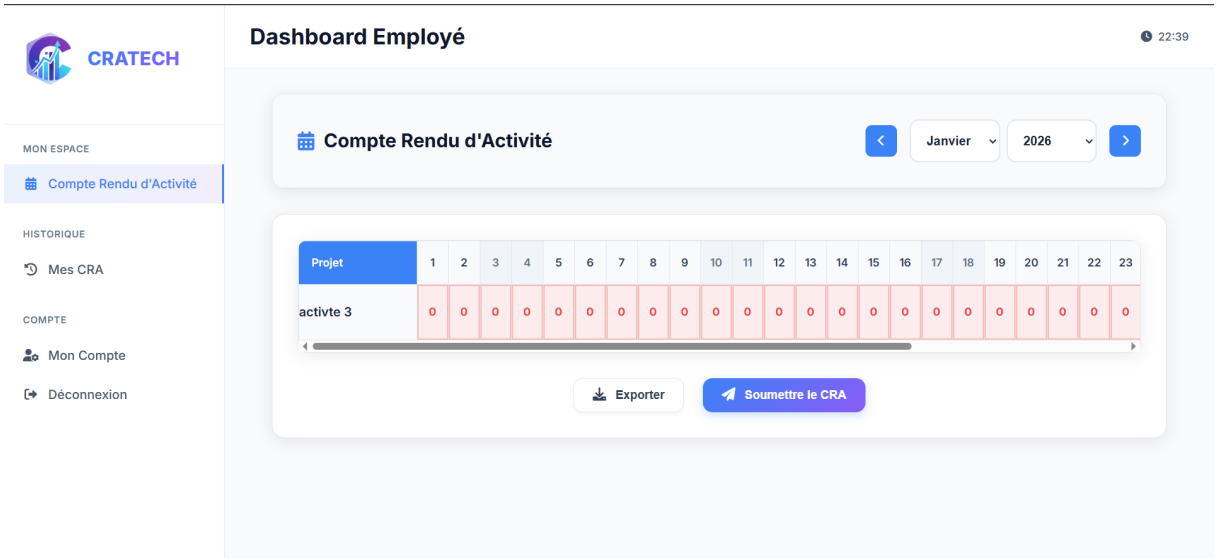


FIGURE 4.4 – Tableau de bord employé et sous-traitant

Historiques des Comptes Rendu d'Activité

Cette interface permet à l'utilisateur de visualiser tout les CRA remplis : soumis ou non sousmis , et suivre leur etat de validation .

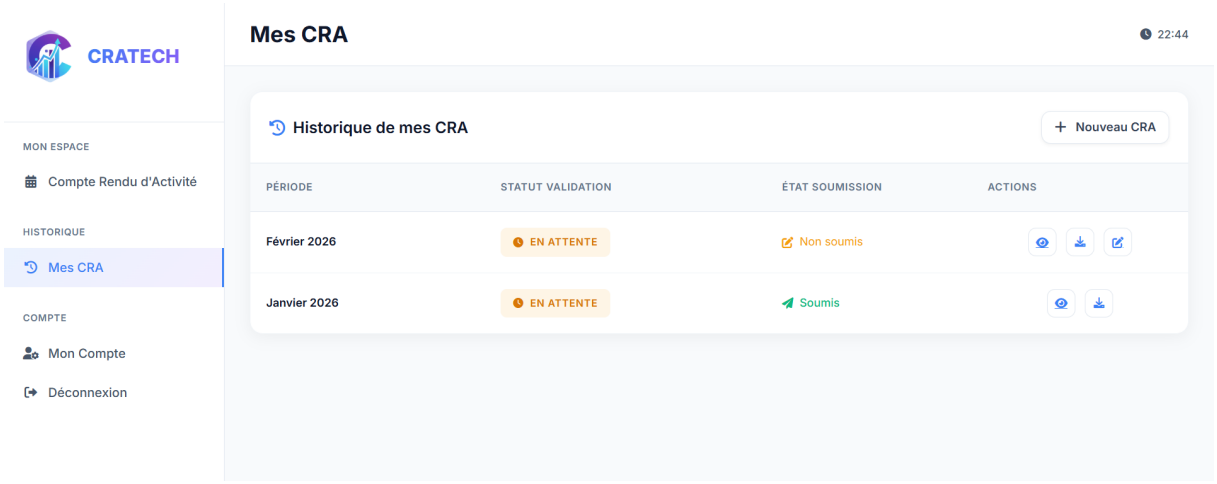


FIGURE 4.5 – Historique des CRA

4.2.2 Interface Administrateur

Tableau de Bord Administrateur

Le tableau de bord administrateur permet de superviser l'ensemble de l'activité du système. Il fournit des indicateurs globaux relatifs aux CRA, aux utilisateurs et aux projets.

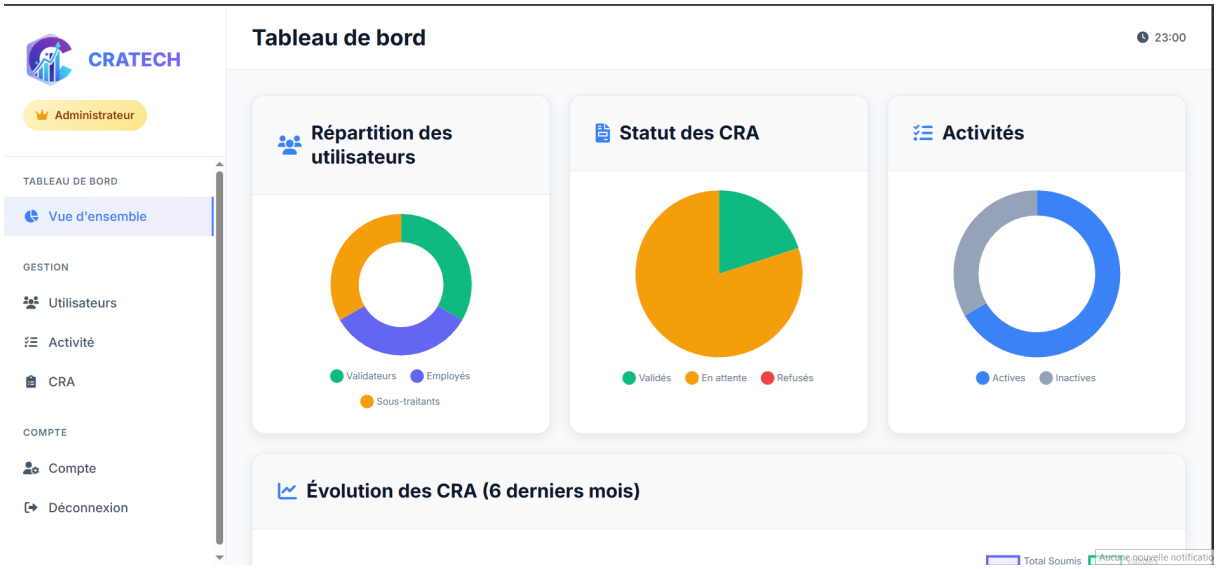


FIGURE 4.6 – Tableau de bord administrateur

Gestion des Utilisateurs

Cette interface permet à l'administrateur de créer, modifier et gérer les comptes utilisateurs.

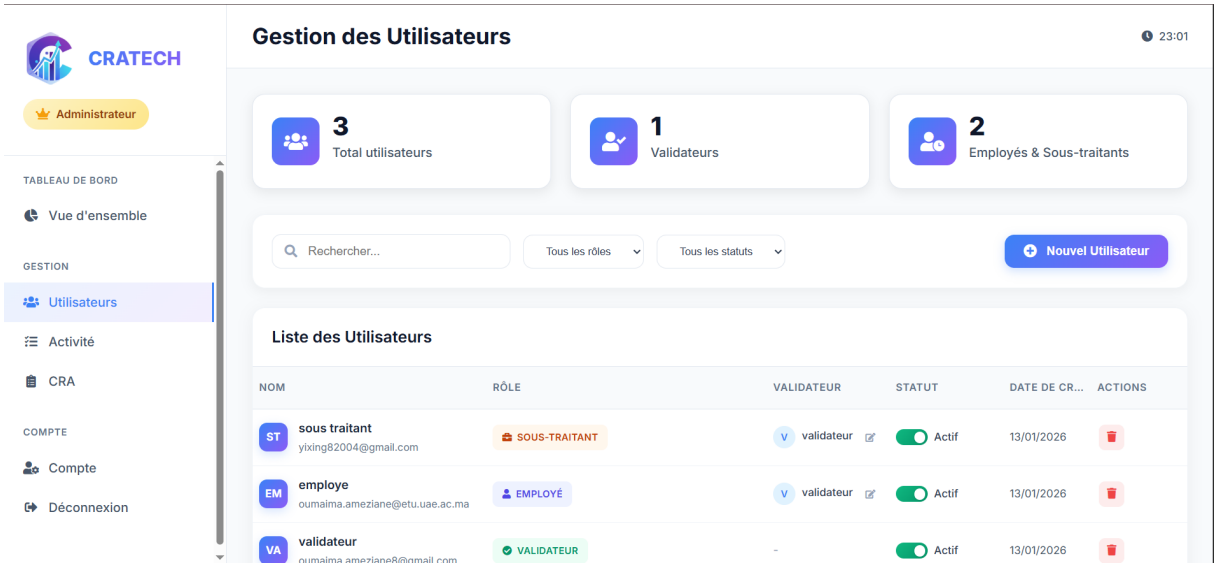


FIGURE 4.7 – Interface de gestion des utilisateurs

Gestion des Activités

Cette interface permet à l'administrateur de créer, modifier et gérer les activités, d'affecter les utilisateurs aux différentes projets.

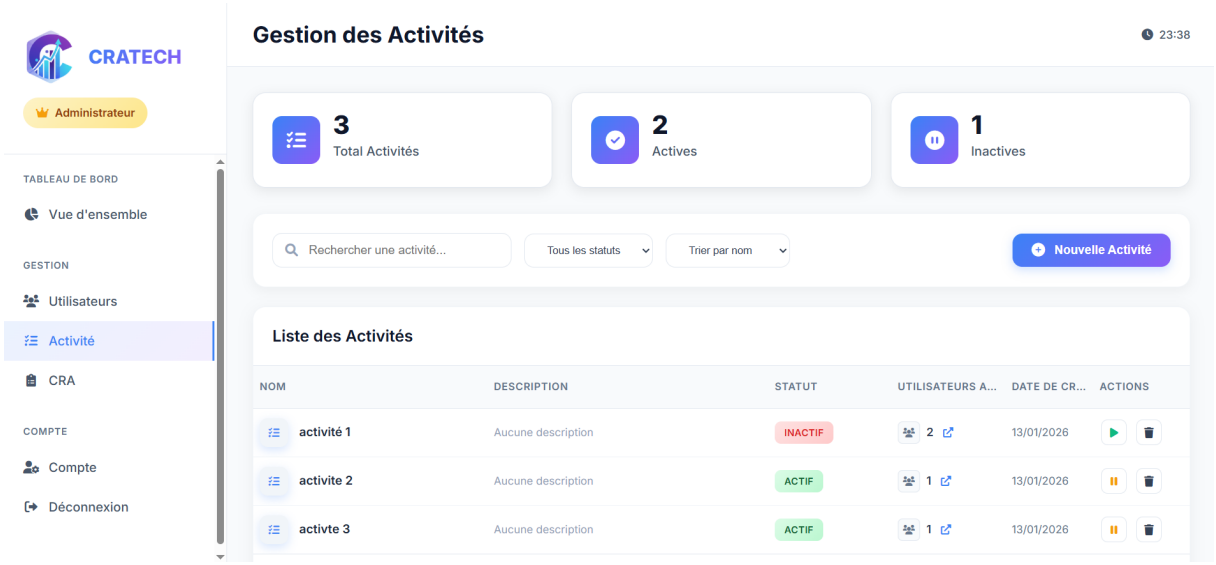


FIGURE 4.8 – Interface de gestion des activites

4.2.3 Interface Validateur

Liste des CRA à Valider

Le validateur accède à une liste des CRA soumis par les utilisateurs. Chaque CRA peut être consulté afin d'en examiner le contenu détaillé.

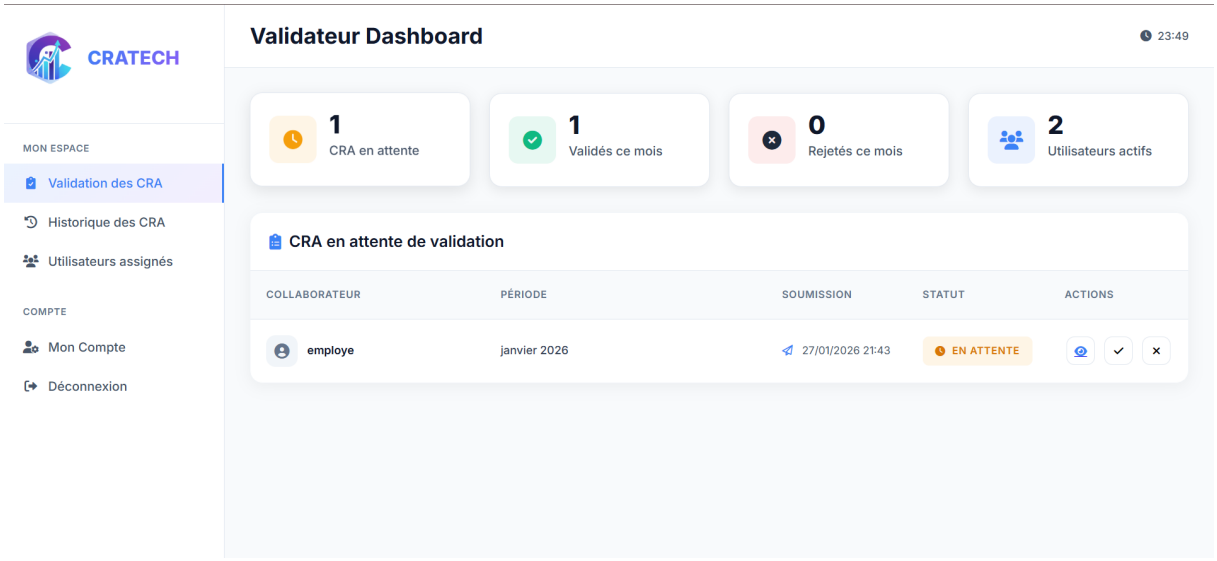


FIGURE 4.9 – Liste des CRA soumis

Validation ou Rejet d'un CRA

Cette interface permet au validateur de visualiser tout l'historique des CRA validés ou refusés pour plus de traçabilité.

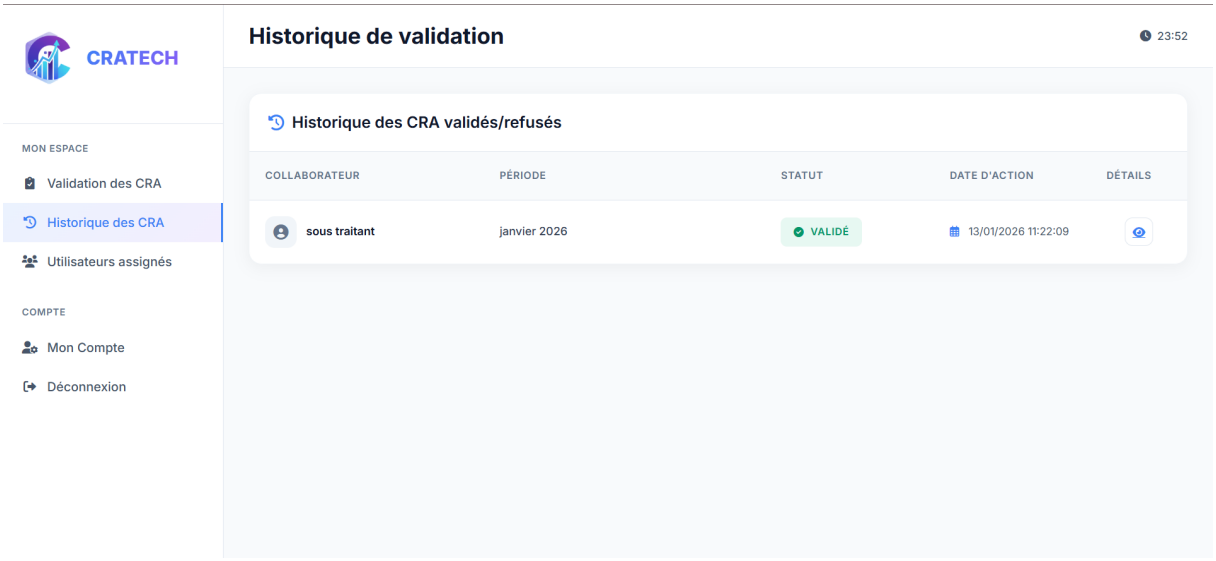


FIGURE 4.10 – Interface de validation des CRA

4.3 Évaluation critique des résultats

4.3.1 Performance de l'architecture technique

L'architecture découplée entre le backend Laravel et le frontend enrichi en JavaScript a permis d'atteindre une fluidité proche d'une application de bureau. La centralisation des styles via un système de variables CSS constitue un atout majeur pour la maintenance, permettant une évolution graphique de l'ensemble de l'application sans modification structurelle. De plus, la gestion « Zéro-Redondance » de la base de données assure une cohérence parfaite entre les données saisies et les rapports exportés.

4.3.2 Impact sur le workflow métier

L'automatisation du processus a eu un impact immédiat sur la productivité :

— **Réduction des erreurs** : Les calculs automatiques des totaux lors de la saisie éliminent les incohérences de reporting.

— **Gain de temps** : Les notifications automatiques réduisent le délai entre la soumission et la validation du CRA de plusieurs jours à quelques heures.

— **Centralisation** : La fin de l'utilisation des fichiers Excel partagés supprime tout risque de perte de données ou de conflits de version.

4.4 Compétences Acquises

4.4.1 Compétences Techniques

Durant ce stage, j'ai développé ou approfondi les compétences techniques suivantes :

Développement Backend :

1. Maîtrise approfondie de Laravel 12.0
2. Conception d'API RESTful
3. Gestion avancée de bases de données avec Eloquent ORM
4. Implémentation de systèmes d'authentification SSO avec Keycloak
5. Sécurité applicative (CSRF, validation, permissions)

Développement Frontend :

1. Utilisation de Vite.js pour le bundling moderne
2. Maîtrise de Tailwind CSS pour le design utilitaire
3. Développement JavaScript moderne (ES6+)
4. Intégration de bibliothèques de visualisation (Chart.js)
5. Création d'interfaces utilisateur responsive

Outils et Méthodologies :

1. Gestion de versions avec Git
2. Utilisation de Composer et npm pour la gestion des dépendances
3. Configuration d'environnements de développement

4.4.2 Compétences Transversales

Les compétences transversales développées incluent :

Gestion de Projet :

1. Priorisation des tâches selon les objectifs
2. Suivi de l'avancement du projet
3. Gestion des imprévus et adaptabilité

Communication :

1. Échanges réguliers avec l'encadrant
2. Présentation de l'avancement du projet
3. Rédaction de documentation technique et utilisateur
4. Explication des choix techniques

Résolution de Problèmes :

1. Analyse et débogage de problèmes techniques
2. Recherche de solutions adaptées
3. Prise de décision technique éclairée
4. Créativité dans l'implémentation

4.5 Conclusion Générale

Ce stage d'initiation a représenté une opportunité exceptionnelle de mettre en pratique les connaissances théoriques acquises durant ma formation en Génie Informatique à l'ENSA Tétouan.

Le projet CRATECH m'a permis de relever un défi technique ambitieux : concevoir et développer une application web complète de gestion des Comptes Rendus d'Activités. De l'analyse des besoins à la réalisation finale, en passant par la conception et les tests, j'ai pu appréhender toutes les phases du cycle de développement logiciel.

L'utilisation d'une pile technologique moderne (Laravel 12.0, Tailwind CSS 4, Vite.js 6, MySQL, Keycloak) m'a confrontée aux standards actuels de l'industrie et m'a permis d'acquérir une expertise solide en développement web full-stack. L'intégration d'un système d'authentification SSO via Keycloak a particulièrement enrichi mes connaissances en matière de sécurité applicative.

Au-delà des aspects purement techniques, ce stage m'a fait découvrir la réalité du monde professionnel : la gestion des priorités et des contraintes de temps et l'importance de la documentation. J'ai appris à faire preuve d'autonomie tout en sachant solliciter de l'aide quand nécessaire.

Les perspectives d'évolution identifiées montrent le potentiel de CRATECH à devenir une solution complète de gestion du temps et des activités. Je suis fière d'avoir contribué à la création de cet outil qui répond à un besoin concret et qui pourra évoluer pour s'adapter aux futurs besoins des utilisateurs.

En conclusion, ce stage a été une expérience formatrice et enrichissante qui a consolidé mes compétences techniques, développé mes capacités professionnelles le domaine du développement d'applications web.

Chapitre 5

Documentation Technique

5.1 Installation et Déploiement

5.1.1 Prérequis

1. PHP 8.2 ou supérieur
2. Composer 2.x
3. Node.js 18+ et npm
4. MySQL 8.0 ou MariaDB 10.6+
5. Serveur Keycloak configuré
6. Serveur web (Apache/Nginx)

5.1.2 Procédure d'Installation

1. Cloner le dépôt Git : https://github.com/oumaima650/CRATECH
2. Installer les dépendances PHP avec Composer
3. Installer les dépendances JavaScript avec npm
4. Configurer les variables d'environnement
5. Générer la clé d'application
6. Exécuter les migrations de base de données
7. Compiler les assets frontend
8. Configurer le serveur web
9. Configurer l'intégration Keycloak

5.1.3 Fichier d'Environnement

```
APP_NAME=CRATECH
APP_ENV=production
APP_KEY=base64:...
APP_DEBUG=false
APP_URL=http://localhost
```

```
6
7  DB_CONNECTION=mysql
8  DB_HOST=127.0.0.1
9  DB_PORT=3306
10 DB_DATABASE=cratech
11 DB_USERNAME=root
12 DB_PASSWORD=
13
14 KEYCLOAK_BASE_URL=https://keycloak.example.com
15 KEYCLOAK_REALM=cratech-realm
16 KEYCLOAK_CLIENT_ID=cratech-client
17 KEYCLOAK_CLIENT_SECRET=your-client-secret
18 KEYCLOAK_REDIRECT_URI=http://localhost/auth/callback
```